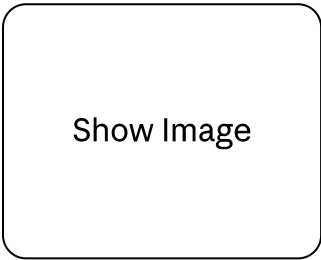


# AWS 3-Tier Architecture

A production-grade, highly available 3-tier web application infrastructure built on Amazon Web Services (AWS). This project demonstrates core AWS networking, compute, load balancing, auto scaling, and database concepts across two Availability Zones in `us-east-1`.

Built as part of a hands-on AWS learning project to complement the AWS Solutions Architect Associate (SAA-C03) certification.

## Architecture Diagram



## Overview

The architecture follows a classic 3-tier design pattern separating the **Web**, **Application**, and **Database** tiers into distinct layers with strict security boundaries between them. All tiers are deployed inside a single VPC across two Availability Zones for high availability.

| Tier             | Purpose                                                | Subnet Type |
|------------------|--------------------------------------------------------|-------------|
| Web Tier         | Serves HTTP traffic to end users                       | Public      |
| Application Tier | Business logic, processes requests from web tier       | Private     |
| Database Tier    | Persistent data storage, accessible only from app tier | Private     |

## Infrastructure Components

### Networking

| Resource        | Name                         | Details                               |
|-----------------|------------------------------|---------------------------------------|
| VPC             | <code>3tier-vpc</code>       | CIDR: <code>10.0.0.0/16</code>        |
| Public Subnet 1 | <code>public-subnet-1</code> | <code>10.0.0.0/20</code> — us-east-1a |

| Resource            | Name             | Details                          |
|---------------------|------------------|----------------------------------|
| Public Subnet 2     | public-subnet-2  | 10.0.16.0/20 — us-east-1b        |
| Private Subnet 1    | private-subnet-1 | 10.0.128.0/20 — us-east-1a (App) |
| Private Subnet 2    | private-subnet-2 | 10.0.144.0/20 — us-east-1b (App) |
| Private Subnet 3    | private-subnet-3 | 10.0.160.0/20 — us-east-1a (DB)  |
| Private Subnet 4    | private-subnet-4 | 10.0.176.0/20 — us-east-1b (DB)  |
| Internet Gateway    | 3tier-igw        | Attached to 3tier-vpc            |
| NAT Gateway         | 3tier-nat-gw     | Deployed in public-subnet-1      |
| Public Route Table  | public-rtb       | Routes 0.0.0.0/0 → IGW           |
| Private Route Table | private-rtb      | Routes 0.0.0.0/0 → NAT Gateway   |

## Web Tier

| Resource                  | Name         | Details                             |
|---------------------------|--------------|-------------------------------------|
| Application Load Balancer | web-tier-alb | Internet-facing, public subnets     |
| Launch Template           | web-tier-lt  | Amazon Linux 2023, t2.micro, Apache |
| Auto Scaling Group        | web-tier-asg | Min: 2, Desired: 2, Max: 4          |
| Target Group              | web-tier-tg  | HTTP:80, health check on /          |
| Bastion Host              | bastion-host | t2.micro in public-subnet-1         |

## Application Tier

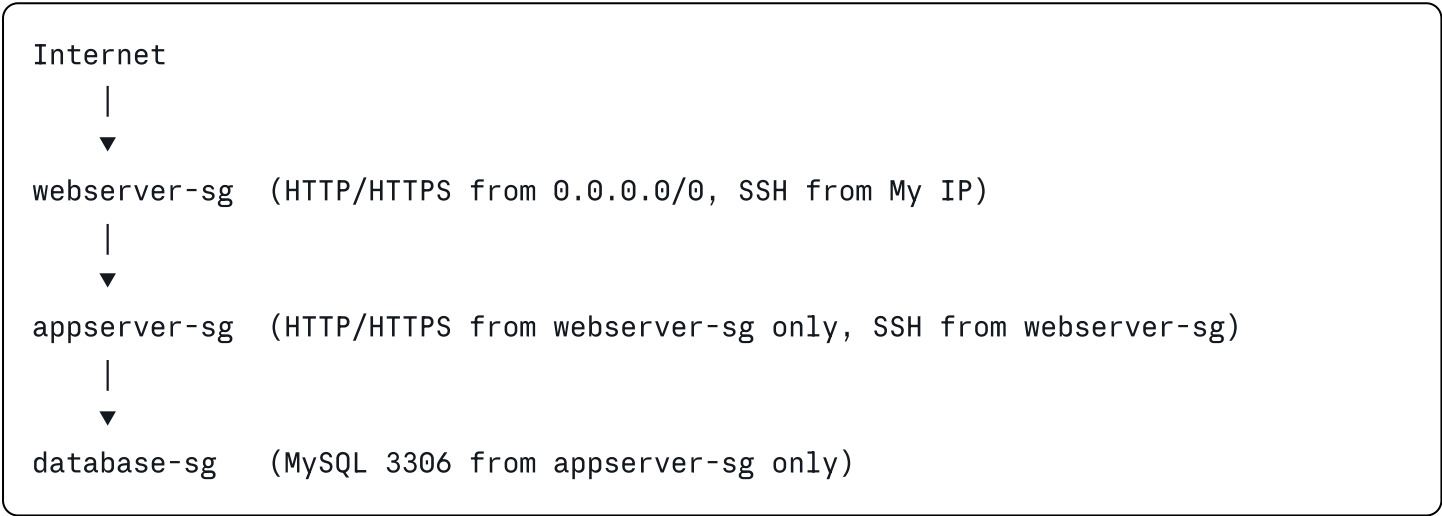
| Resource                  | Name         | Details                                         |
|---------------------------|--------------|-------------------------------------------------|
| Application Load Balancer | app-tier-alb | Internal, private subnets                       |
| Launch Template           | app-tier-lt  | Amazon Linux 2023, t2.micro, Python HTTP server |
| Auto Scaling Group        | app-tier-asg | Min: 2, Desired: 2, Max: 4                      |
| Target Group              | app-tier-tg  | HTTP:80, health check on /                      |

## Database Tier

| Resource        | Name                               | Details                             |
|-----------------|------------------------------------|-------------------------------------|
| RDS Instance    | <code>tier-db</code>               | MySQL 8.0, db.t3.micro, Single-AZ   |
| DB Subnet Group | <code>db-subnet-group-3tier</code> | private-subnet-3 & private-subnet-4 |
|                 |                                    |                                     |
|                 |                                    |                                     |

## Security Groups

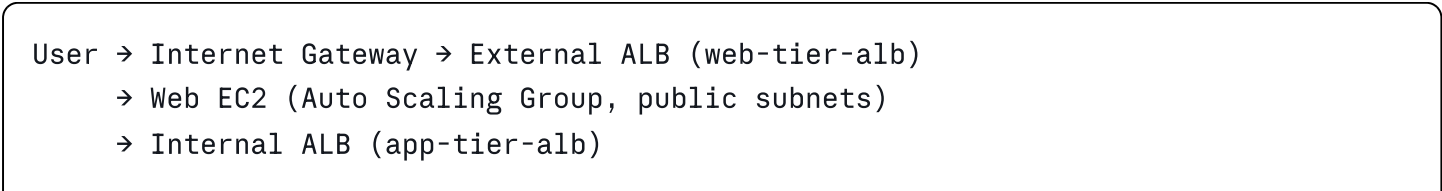
Security is implemented using a layered approach — each tier only accepts traffic from the tier directly above it. Nothing reaches the database except the application tier.



| Security Group            | Inbound Rules                                                            | Purpose                          |
|---------------------------|--------------------------------------------------------------------------|----------------------------------|
| <code>webserver-sg</code> | HTTP (80), HTTPS (443) from <code>0.0.0.0/0</code> ; SSH (22) from My IP | Web EC2 instances & external ALB |
| <code>appserver-sg</code> | HTTP (80), HTTPS (443), SSH (22) from <code>webserver-sg</code> only     | App EC2 instances & internal ALB |
| <code>database-sg</code>  | MySQL (3306) from <code>appserver-sg</code> only                         | RDS instance                     |

## Traffic Flow

### Inbound (User Request)



- App EC2 (Auto Scaling Group, private subnets)
- RDS MySQL (private subnets)

Outbound (Private EC2s reaching internet)

App EC2 (private subnet) → NAT Gateway (public subnet) → Internet Gateway → Internet

Outbound only — no inbound connections can be initiated from the internet to private instances.

Tagging Strategy

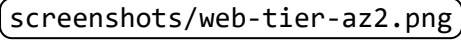
All resources are tagged consistently for cost tracking, resource management, and governance.

| Tag Key     | Tag Value                      |
|-------------|--------------------------------|
| Project     | 3TierArchitecture              |
| Environment | Dev                            |
| ManagedBy   | Manual                         |
| Owner       | Ola                            |
| CostCenter  | Learning                       |
| Tier        | Network / Web / App / Database |

Tags were applied using **AWS Resource Groups & Tag Editor** for bulk tagging efficiency. The **Tier** tag enables filtering resources by architectural layer for both management and cost reporting.

Screenshots

| Screenshot                | Description                     |
|---------------------------|---------------------------------|
| screenshots/vpc.png       | 3tier-vpc with 10.0.0.0/16 CIDR |
| screenshots/subnets.png   | All 6 subnets across 2 AZs      |
| screenshots/instances.png | All EC2 instances running       |

| Screenshot                                                                       | Description                      |
|----------------------------------------------------------------------------------|----------------------------------|
|  | Both ALBs active                 |
|  | RDS instance available           |
|  | Web tier serving from us-east-1a |
|  | Web tier serving from us-east-1b |

## Key Design Decisions

**Why NAT Gateway in a public subnet?** Private EC2 instances need outbound internet access (e.g. for `yum update`) but must not be directly reachable from the internet. The NAT Gateway sits in the public subnet with an Elastic IP, translating outbound requests from private instances while blocking unsolicited inbound traffic.

**Why two ALBs?** The external ALB handles internet-facing traffic and distributes it across web EC2s. The internal ALB decouples the web and application tiers — web servers don't communicate directly with app servers. This means each tier can scale independently and the app tier is never directly exposed.

**Why a Bastion Host?** App and DB tier instances have no public IP. The bastion host in the public subnet acts as a secure SSH jump server — you SSH into the bastion first, then SSH from there into private instances. This is standard practice for production VPC environments.

**Why Single-AZ RDS?** Multi-AZ RDS runs a synchronous standby replica in a second AZ with automatic failover, roughly doubling cost. For a learning/dev environment, Single-AZ is appropriate. In production, Multi-AZ would be used for the database tier.

---

## Technologies Used

- **Amazon VPC** — Networking, subnets, route tables
  - **Amazon EC2** — Web and application servers (t2.micro)
  - **Elastic Load Balancing** — Application Load Balancers (external & internal)
  - **Auto Scaling** — Automatic EC2 scaling based on demand
  - **Amazon RDS** — Managed MySQL 8.0 database
  - **AWS NAT Gateway** — Outbound internet for private instances
  - **AWS Resource Groups & Tag Editor** — Bulk resource tagging
-

# Region & Availability Zones

| Setting             | Value                                |
|---------------------|--------------------------------------|
| Region              | <code>us-east-1</code> (N. Virginia) |
| Availability Zone 1 | <code>us-east-1a</code>              |
| Availability Zone 2 | <code>us-east-1b</code>              |

## Certification Context

This project was built as a practical demonstration of AWS SAA-C03 exam concepts including:

- VPC design and multi-AZ architecture
- Public vs private subnet patterns
- Security group layering and least-privilege access
- Load balancer types and use cases
- Auto Scaling groups and launch templates
- RDS deployment and subnet groups
- NAT Gateway vs NAT Instance trade-offs
- AWS tagging best practices for cost allocation and governance

## Author

Ola AWS Solutions Architect Associate (SAA-C03)